

# Project Design Document

**Project Name:** Odyssey - Real-Time Public Transit Analytics Platform

**Author:** Nanda kumar Bandi - Data Engineer

## 1. Executive Summary:

Project Odyssey aims to build a cloud-native data platform for a city's transit authority. The platform will ingest, process, and analyze real-time vehicle location data and historical operational data. The end goal is to provide live operational monitoring via interactive dashboards and a robust data warehouse to enable business intelligence and performance analytics, ultimately improving service efficiency and passenger experience.

## 2. Business Problem Statement:

The City Transit Authority (CTA) currently faces significant operational challenges due to a lack of timely and accessible data. There key pain points:

- **Reactive Operations:** Dispatchers and managers lack a real-time view of the entire fleet, making it difficult to respond to service disruptions, vehicle bunching, or accidents proactively.
- **Poor On-Time Performance (OTP):** Schedules are based on ideal conditions. Without historical performance data, the CTA cannot identify chronic bottlenecks or adjust schedules realistically, leading to passenger dissatisfaction.
- **Data Silos:** Operational data is fragmented across different systems, making it impossible for analysts to get a holistic view of performance or answer complex questions about the interplay between routes, schedules, and external factors like weather.
- **Inefficient Resource Allocation:** Without clear data on passenger load and route performance, decisions about fleet allocation and schedule frequency are based on anecdote rather than evidence.

## 3. Proposed Solution:

We will design and implement an end-to-end, scalable data platform that will serve as the single source of truth for all transit operations data. The platform will deliver:

- A **Live Operations Dashboard** showing real-time vehicle positions on a map, with key metrics like speed and schedule adherence.
- An **Analytical Data Warehouse** with clean, modeled data, enabling BI analysts to build dashboards for tracking Key Performance Indicators (KPIs) like OTP, average route speed, and vehicle utilization.
- An **automated and orchestrated pipeline** that is reliable, scalable, and requires minimal manual intervention.

#### 4. Data Sources Info:

- **Primary (Real-time): GTFS-Realtime (GTFS-RT):**

**Description:** A live data feed providing vehicle positions, trip updates, and service alerts.

**Format:** Protocol Buffers (Protobuf) over HTTP. This is a binary format, requiring deserialization.

**Source:** We will use the live feed from a public transit authority. A great option for this project is the Bengaluru Metropolitan Transport Corporation (BMTC) feed, which provides city-wide bus data.

- **Primary (Reference): GTFS-Static:**

**Description:** A set of CSV files (routes.txt, stops.txt, trips.txt, stop\_times.txt) that provides the schedule and geographical information for the transit system. This data gives context to the real-time feed.

**Format:** ZIP archive containing multiple .txt (CSV) files.

**Source:** The corresponding static feed from the same transit authority (e.g., BMTC's static data).

- **Secondary (Enrichment): Weather Data:**

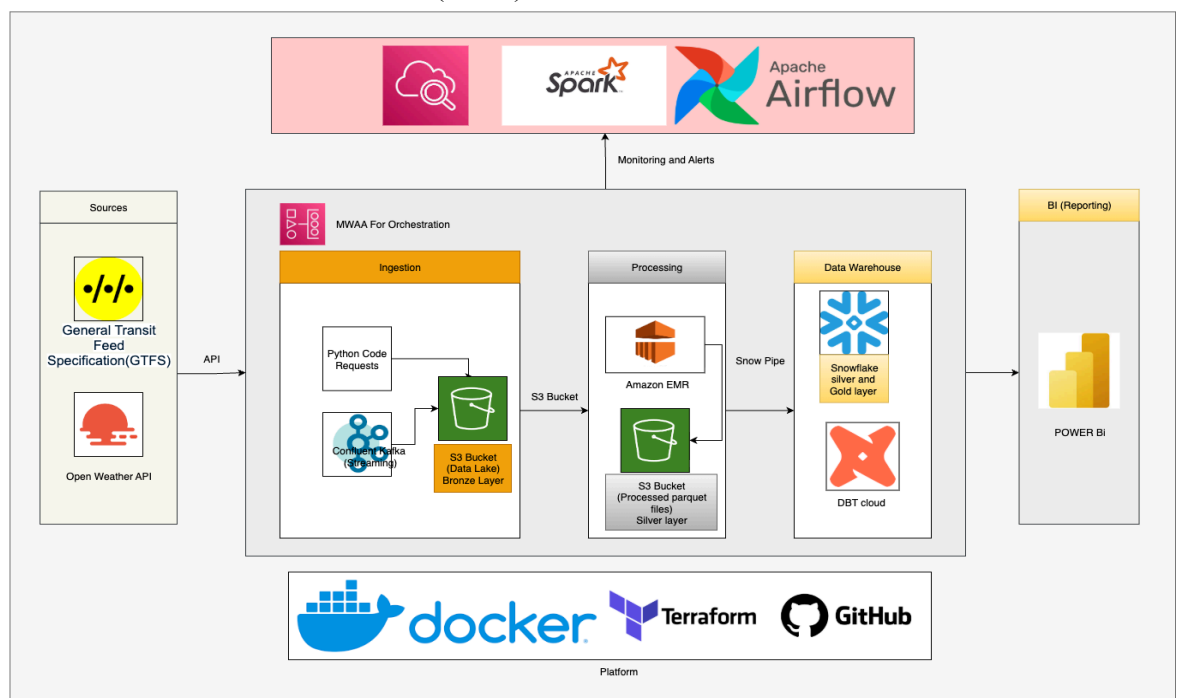
**Description:** Historical and current weather conditions (e.g., rain, temperature, visibility). This will allow us to analyze the impact of weather on transit performance.

**Format:** JSON.

**Source:** A public weather API like OpenWeatherMap, which has a generous free tier. We will poll this API periodically.

#### 5. System Design & Technology Stack:

Our design philosophy is to use best managed services where possible to accelerate development and ensure scalability, while leveraging open-source standards. We will build this on Amazon Web Services (AWS).



| component                   | Technology               |
|-----------------------------|--------------------------|
| Cloud Provider              | Amazon Web services(AWS) |
| Infrastructure as code(IAC) | Terraform                |
| Data Lake Storage           | Amazon S3                |
| Streaming Ingestion         | Confluent Cloud          |
| Processing Engine           | Amazon EMR               |
| Data Warehouse              | Snowflake                |
| Data Transformation         | DBT cloud                |
| Orchestration               | Amazon MWAA              |
| BI & Visualisation          | Power BI                 |
| Containerization            | Docker                   |

## 6. Data Structure & Modeling:

- **Source Data Structure:**

**GTFS-RT (Protobuf):** A nested, binary structure. The key entity is FeedMessage, which contains multiple FeedEntity objects. Each FeedEntity can contain a TripUpdate, a VehiclePosition, or a ServiceAlert. We will primarily focus on VehiclePosition.

**GTFS-Static (CSV):** A collection of relational tables. For example, routes.txt has route details, and stops.txt has stop locations. They are linked by keys like route\_id and stop\_id.

**Weather API (JSON):** A nested JSON object containing weather metrics for a specific location and time.

- **Data Lake Architecture (Medallion):**

We will structure our S3 data lake using the Bronze-Silver-Gold (Medallion) architecture.

**Bronze Layer (s3://.../bronze/):** Raw, immutable data exactly as it arrived from the source. (Raw Protobuf files, raw JSON from weather API, raw GTFS zip).

**Silver Layer (s3://.../silver/):** Cleaned, partitioned, and enriched data stored in an analytical format. (Vehicle positions joined with route information, stored as Apache Parquet, partitioned by date).

**Gold Layer (s3://.../gold/):** Business-level aggregations and tables ready for analytics. (e.g., hourly on-time performance summaries per route).

- **Data Warehouse Modeling (Star Schema)**

In Snowflake, we will use dbt to build a classic Star Schema for intuitive analytical querying.

**Fact Table:** fct\_vehicle\_positions

**Grain:** One row per vehicle position update received from the feed.

**Columns:** position\_timestamp, vehicle\_id, route\_id, stop\_id, latitude, longitude, speed\_mps, delay\_seconds, weather\_condition\_key, etc.

**Dimension Tables:**

**dim\_routes:** Details on each route (route\_short\_name, route\_long\_name, etc.). Sourced from routes.txt.

**dim\_vehicles:** Details on each vehicle (if available, or can be inferred).

**dim\_stops:** Details on each stop (stop\_name, stop\_lat, stop\_lon). Sourced from stops.txt.

**dim\_dates:** A pre-generated date dimension for easy time-based analysis (date, day\_of\_week, month, is\_holiday, etc.).

7. Managing Data Access:

- We use **AWS Identity and Access Management (IAM)** to control which *services* can talk to each other. We will create specific IAM Roles for our services.
- This is where we manage access for people. Snowflake has a powerful and granular Role-Based Access Control (RBAC) system.
- How we'll use it: We will create roles for different business functions, for example, operations\_analyst, finance\_analyst, and data\_scientist.

8. Monitoring and Alerts:

- Airflow is our central command center for monitoring. The Airflow UI gives us a visual representation of all our DAG runs. We can immediately see which tasks have failed (they turn red).
- Logs: We can access the complete logs for every single task directly within the Airflow UI. This is our first stop for debugging.
- Alerting: Airflow has built-in alerting capabilities. We will configure our DAGs to automatically send an email or a Slack message (a more common practice) whenever a critical task fails.
- CloudWatch is the monitoring service for all of AWS. It collects metrics, logs, and events. We can set up CloudWatch Alarms to notify us of infrastructure-level problems.
- Sometimes the pipeline runs successfully, but the data itself is wrong. This is a silent failure. Dbt is our primary defense here. We will run a dbt test as a task in our Airflow DAG after our models are built. If a data quality test fails (e.g., a primary key is no longer unique), the Airflow task will fail, and our alert will be triggered, preventing bad data from ever reaching the business users.